

Multimodelling Developement System

UNIT -2

Contents -

- Modelling Paradigms and Techniques: Continuous-time and discrete- event modeling
- System dynamics and agent-based modeling
- Co-simulation concepts and frameworks
- Model validation, verification, and consistency checking.

Continuous-time simulation

- Continuous-time simulation is a type of simulation that continuously tracks the state of the target system.
- Differential equations often model systems in continuous-time simulation. These differential equations define how the state of the system changes over time.
- Continuous-time simulations run on computers are rarely truly continuous-time simulations. Instead, digital computers perform continuous-time simulation approximately. There are two reasons for this:

Continuous-time simulation

- ✓ Real numbers cannot be represented faithfully because they are uncountable.
- ✓ Differential equations can only be solved approximately through some form of discretization.

Discretization in simulation converts continuous variables, time, or space into discrete, finite points, enabling computer-based numerical modeling of complex physical systems. Key methods include finite difference (FDTD), finite element (FEM), and finite volume (FVM). It directly impacts computational speed and accuracy, balancing mesh density and time-step sizes.

Continuous-time simulation

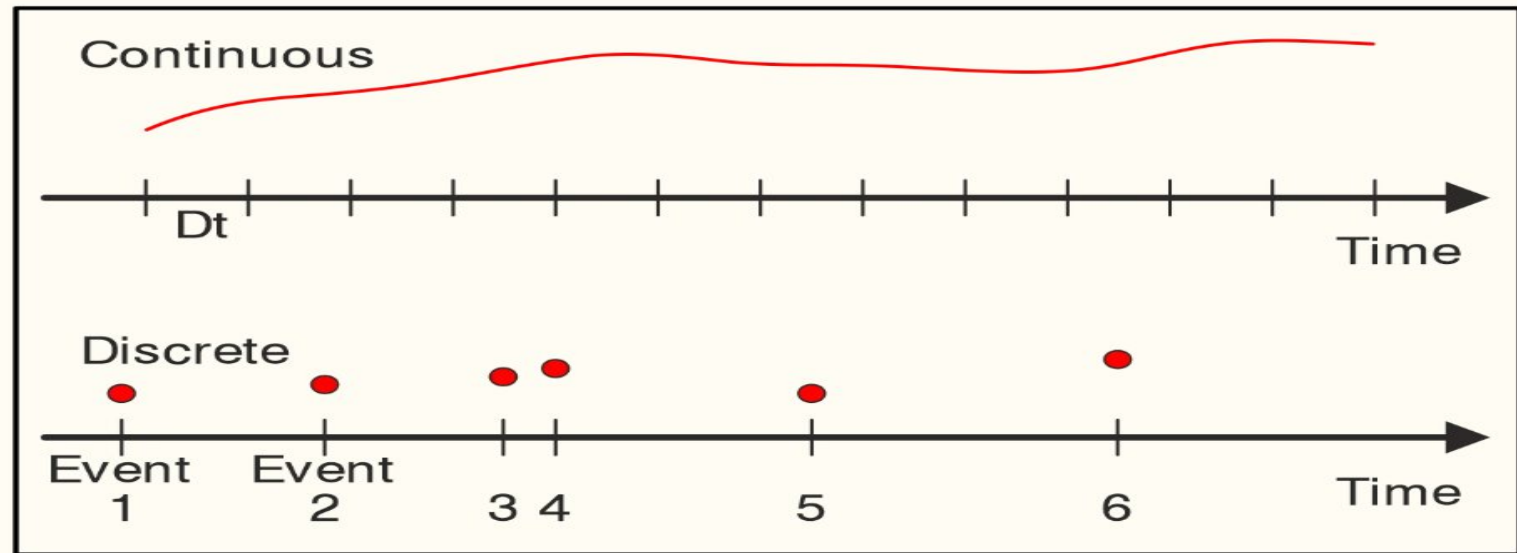


Figure : Updated State Over Simulated Time in Continuous and Discrete Simulation

Continuous-time simulation

- ✓ Due to this, digital computers run continuous-time simulations by dividing (discretizing) time into small time steps, which is an approach called fixed-increment time progression. In some advanced cases, variable-increment time progression can be used.

Core Aspects of Discretization -

- ✓ **Spatial Discretization (Meshing):** Dividing a continuous geometric domain into smaller, non-overlapping cells (mesh) to calculate parameters like stress, temperature, or pressure at specific points.
- ✓ **Temporal Discretization (Time-Stepping):** Breaking continuous time into distinct steps, such as Δt , used in dynamic simulations for transient analysis.
- ✓ **Mathematical Discretization:** Converting continuous differential equations into algebraic equations that digital computers can solve (e.g., Euler, Tustin's methods).

Common Techniques -

- **Finite Difference Method (FDM):** Replaces continuous derivatives with differences between mesh points.
- **Finite Element Method (FEM):** Used for solid mechanics and heat transfer by dividing structures into smaller elements.
- **Finite Volume Method (FVM):** Commonly used in Computational Fluid Dynamics (CFD) to conserve quantities within a volume, such as using first-order or second-order upwind schemes.

Continuous-time vs discrete-event simulation

Discrete-event simulation is a significant alternative to continuous-time simulation, although many other types of simulation modeling exist.

At a high level, the primary differences between continuous-time and discrete-event simulations are listed in Table

Continuous-time vs discrete-event simulation

	Continuous-Time Models	Discrete-Event Models
System Being Simulated	Specific mechanical unit with complex functioning	Multiple, often simple, entities
Definition of System State	Distribution of energy within system	Categorization of current system properties
Typical Measures of System State	Position, attitude, velocity (deterministic)	Queue size, incidence (statistical properties thereof)
Typical Factor Driving Updates	Time	Transitions from state to state
Capability of Simulation	Analyze deterministic dynamic behavior of a mechanical unit	Analyze stochastic nature of interactions between entities
Common Uses	Design and analysis of unit, (real-time) training	Analysis and planning of operations with multiple entities

Table : Comparison of Continuous-Time and Discrete-Event Models

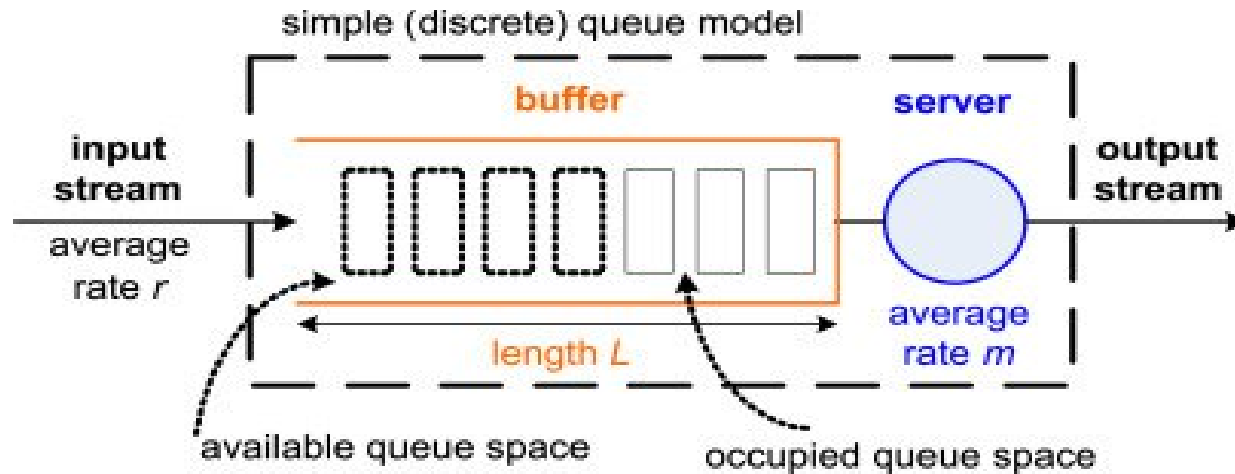
The following main characteristics of DES(Discrete Event Simulation):

- Events occur at identifiable points in time.
- DES emphasizes events and state changes rather than the passage of time itself.
- DES uses time skipping – that is, it jumps between changes in the system, ignoring the state in between.
- DES often relies on queueing theory and uses different queueing systems to model a wide array of processes and phenomena.

Queuing Theory -

- Queuing Theory is the mathematical study of waiting lines (queues) to analyze and optimize systems where demand exceeds service capacity, focusing on factors like arrival rates, service times, and queue length to improve efficiency in areas like call centers, traffic, and computer networks.
- It uses models to predict wait times, understand congestion, and make business decisions about resource allocation, considering components like customers (people, data packets), servers, arrival/service processes, and queuing discipline (e.g., FIFO).

Queuing Theory -



Queing Theory -

- ✓ Customers/Entities: People, data packets, or jobs needing service.
 - ✓ Servers: The resources providing the service (tellers, network routers, printers).
 - ✓ Arrival Process: How often customers arrive (arrival rate, λ).
 - ✓ Service Process: How long service takes (service rate, μ).
 - ✓ Queue Discipline: Rules for serving customers (e.g., First-In, First-Out (FIFO), Last-In, First-Out (LIFO)).
- System Capacity: Maximum number of customers allowed.

Continuous-time simulation directly contrasts these qualities. Specifically :

- ✓ Continuous-time simulation emphasizes the continuous change of the target system over time.
- ✓ Continuous-time simulation constantly tracks time and can produce data even when no changes occur in the target system.
- ✓ Continuous-time simulation is typically applied to natural sciences phenomena - like biological, chemical, and environmental processes. These applications are less likely to involve discrete queuing behaviors.

Another difference between continuous-time and discrete-event simulations is their adaptability to both continuous and discrete problems -

- ✓ Continuous-time simulation can be used to model continuous and discrete processes. Although discrete processes are not continuous by nature, continuous-time models can successfully model them and generate valid results. However, a continuous-time model would require more time and computation for the same discrete problem. Additionally, for some discrete systems, the output of a continuous-time simulation model might not make sense.
- ✓ Discrete models could, in theory, be applied to continuous-time systems as well. However, a discrete-event model would provide a less refined insight into a continuous-time system because of its nature.

System Dynamics

- ✓ The System Dynamics Model is a methodology for understanding, modeling, and analyzing complex systems.
- ✓ It focuses on the feedback loops, time delays, and accumulations that drive system behavior. The model helps identify the underlying structure of systems and predict the impact of changes over time.
- ✓ Key components of the System Dynamics Model include:

System Dynamics

- **Causal Loop Diagrams:** Visual representations of the feedback loops and causal relationships within a system.
- **Stock and Flow Diagrams:** Diagrams that show the accumulations (stocks) and the rates of change (flows) within a system.
- **Simulation Models:** Computational models that simulate the behavior of the system over time, allowing for experimentation and analysis.

System Dynamics

- ✓ The distinctive feature of the System Dynamics Model is its ability to capture the complexity and dynamics of systems through feedback loops and time delays.
- ✓ This approach provides a deep understanding of how systems behave over time and how different variables interact, enabling more effective decision-making and problem-solving.

Situations in which System Dynamics used -

- ✓ Complex problem-solving and decision-making
- ✓ Strategic planning and scenario analysis
- ✓ Policy design and evaluation
- ✓ Organizational and systems change
- ✓ Any context requiring a deep understanding of dynamic and complex systems

Applications -

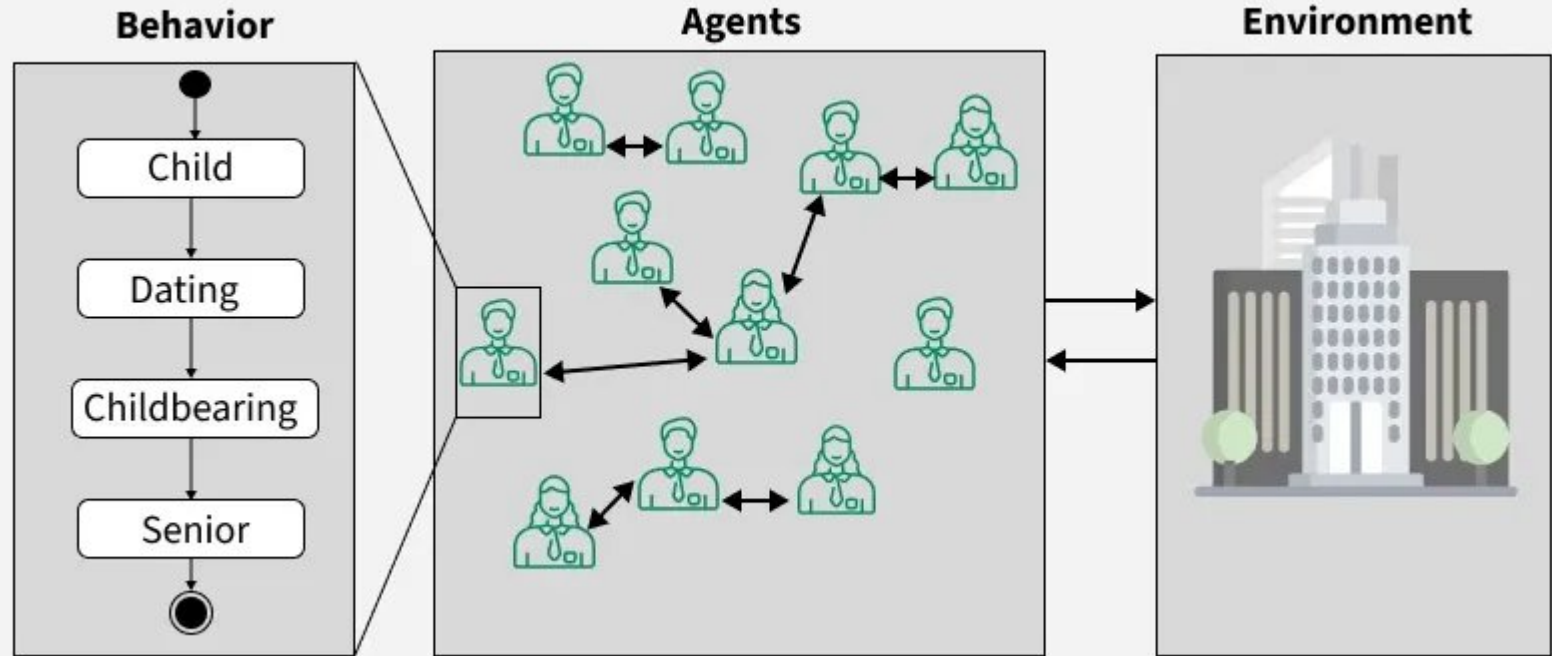
- Telecommunications: Managing call centers, network traffic.
- Transportation: Traffic light optimization, airport check-in.
- Manufacturing: Production line efficiency.
- Computer Science: Data flow in networks, print queues.

Goal -

To balance service quality (short waits) with operational costs, finding the optimal number of servers or system design to handle demand efficiently.

Agent Based Modelling

- Agent based modelling is a class of computational models that simulate the behaviour of autonomous agents such as people, animals, vehicles or organizations, or even cells.
- Each agent operates according to a set of rules and interacts with other agents and their environment and then the model then tracks how these local interactions give rise to global patterns.



Agents -

- ✓ In agent based modelling (ABM) agents are autonomous, decision making entities that simulate the behavior of individuals components within a system.
- ✓ Each agent operates according to a set of rules that govern its behavior, decision making processes and interactions with other agents and the environment.
- ✓ Agents are typically heterogeneous as they can differ in attributes such as age, preferences, goals, location or memory.

Agents -

- ✓ Agents can be reactive responding to change in their environment or proactive acting with intention or purpose based on internal states or goals.
- ✓ Some models include learning agents which adapt their behavior over time using mechanisms like reinforcement learning, genetic algorithms or evolutionary strategies.

Proactive vs. Reactive Agents

- ✓ **Reactive Agents:** Wait for a specific trigger (e.g., a command) and give simple, immediate responses (like a basic thermostat or older chatbots).
- ✓ **Proactive Agents:** Act in anticipation, learn continuously, and handle complex, multi-step tasks, requiring more data and resources but offering deeper assistance (e.g., modern AI assistants, self-driving cars)

Proactive Agents -

- ✓ Proactive agents are advanced AI systems that anticipate user needs and initiate actions or suggestions without being explicitly prompted, moving beyond simple reactive responses by using context, history, and predictive models to offer timely assistance, like a smart assistant warning about traffic before you leave or an e-commerce site recommending products based on your browsing patterns.
- ✓ They learn from data to predict future states, optimize workflows, and act autonomously to solve potential problems, making them powerful tools for personalized experiences and complex task management.

Agents in AI

- ✓ An artificial intelligence (AI) agent is a software program that can interact with its environment, collect data and use that data to perform self-directed tasks that meet predetermined goals. Humans set goals, but an AI agent independently chooses the best actions it needs to perform to achieve those goals.
- ✓ For example, consider a customer support AI agent that wants to resolve customer queries. The agent will automatically ask the customer different questions, look up information in internal documents and respond with a solution. Based on the customer responses, it determines if it can resolve the query itself or pass it on to a human.

Key Features of AI Agents

- ✓ Autonomous: Act without constant human input and decide next steps from past data like a bookstore bot flags missing invoices.
- ✓ Goal-driven: Optimize for defined objectives like a logistics AI balancing speed, cost and fuel use.
- ✓ Perceptive: Gather info from sensors, inputs or APIs like a cybersecurity agent tracking new threats.

Key Features of AI Agents

- ✓ **Adaptable:** Adjust strategies when situations change.
- ✓ **Collaborative:** Work with humans or other agents toward shared goals like healthcare agents coordinating with patients and doctor.

Components of Agent based models

- ✓ Agents: Independent entities with their own attributes and behavioral rules.
- ✓ Environment: The space where agents exist and interact.
- ✓ Rules of Interaction: Guidelines that define how agents interact with each other and their surroundings.
- ✓ Emergence: The system level patterns or outcomes that arise from simple interactions among agents.

Importance of Agent based models

- ✓ Captures Complexity: Agent based modeling is a technique used for representing complex systems where things don't follow a straight path. In such systems individual agents interact in ways that create nonlinear effects, feedback loops, and adaptive behavior.
- ✓ Bottom up approach: Instead of assuming overall system behavior from the top ABM works from the bottom up. It starts by modelling the individual behaviors of agents. As these agents interact, the overall system behavior emerges naturally.

Importance of Agent based models

- ✓ **Flexibility:** ABM is highly flexible as it can be used to model both spatial systems and non spatial systems and it supports simulations over both discrete time steps and continuous time which makes it suitable for a wide range of fields and problems.
- ✓ **Scenario Testing:** ABM allows you to run 'what if' scenarios which helps researchers, businesses, and policymakers test decisions in a virtual environment before applying them in the real world which saves time, money and reduce risk.

Challenges in ABM

- ✓ **Data requirements:** one of the biggest challenges in creating an effective ABM is the need for detailed and accurate data about how individual agents behave as it works best when it can simulate how people, machines or organizations act in real life.
- ✓ **Computational intensity:** agent based models with thousands or millions of agents can be computationally demanding as every agent acts individually and interacts with others the system must keep track of a lot of variables at once which requires powerful hardware and can take a long time to run simulations.
- ✓ **Design sensitivity:** agent based models can be very sensitive to small changes in design and the model needs to be tested and interpreted carefully. Otherwise small modelling errors could lead to wrong conclusions.

AI Agent Classification

An agent is a system designed to perceive its environment, make decisions and take actions to achieve specific goals. Agents operate autonomously, without direct human control and can be classified based on their behavior, environment and number of interacting agents.

- ✓ Reactive Agents: Respond to immediate environmental stimuli without foresight or planning.
- ✓ Proactive Agents: Anticipate future states and plan actions to achieve long-term goals.
- ✓ Single-Agent Systems: One agent solves a problem independently.
- ✓ Multi-Agent Systems: Multiple agents interact, coordinate or compete to achieve goals; may be homogeneous (similar roles) or heterogeneous (diverse roles).
- ✓ Rational Agents: Choose actions to maximize expected outcomes using both current and historical information.

How do AI Agents Work?

1. Persona:

Each agent is given a clearly defined role, personality and communication style along with specific instructions and descriptions of the tools it can use. A well-crafted persona ensures the agent behaves consistently and appropriately for its role, while also evolving as it gains experience and engages with users or other systems.

2. Memory: Agents typically have multiple types of memory:

- ✓ Short-term memory for the current interaction
- ✓ Long-term memory for storing historical data and conversations
- ✓ Episodic memory for recalling specific past events
- ✓ Consensus memory for sharing knowledge among multiple agents
- ✓ Memory enables an agent to keep context, learn from experience and adapt its behaviour over time.

How do AI Agents Work?

3. Tools:

These are functions or external resources the agent can use to access information, process data, control devices or connect with other systems. Tools may involve physical interfaces, graphical UIs or programmatic APIs. Agents also learn how and when to use these tools effectively, based on their capabilities and context.

4. Model:

Agents use large language model (LLM) which serves as the agent's "brain". The LLM interprets instructions, reasons about solutions, generates language and orchestrates other components including memory retrieval and tools to use to carry out tasks.

Architecture of AI Agents

There are four main components in an AI agent's architecture:

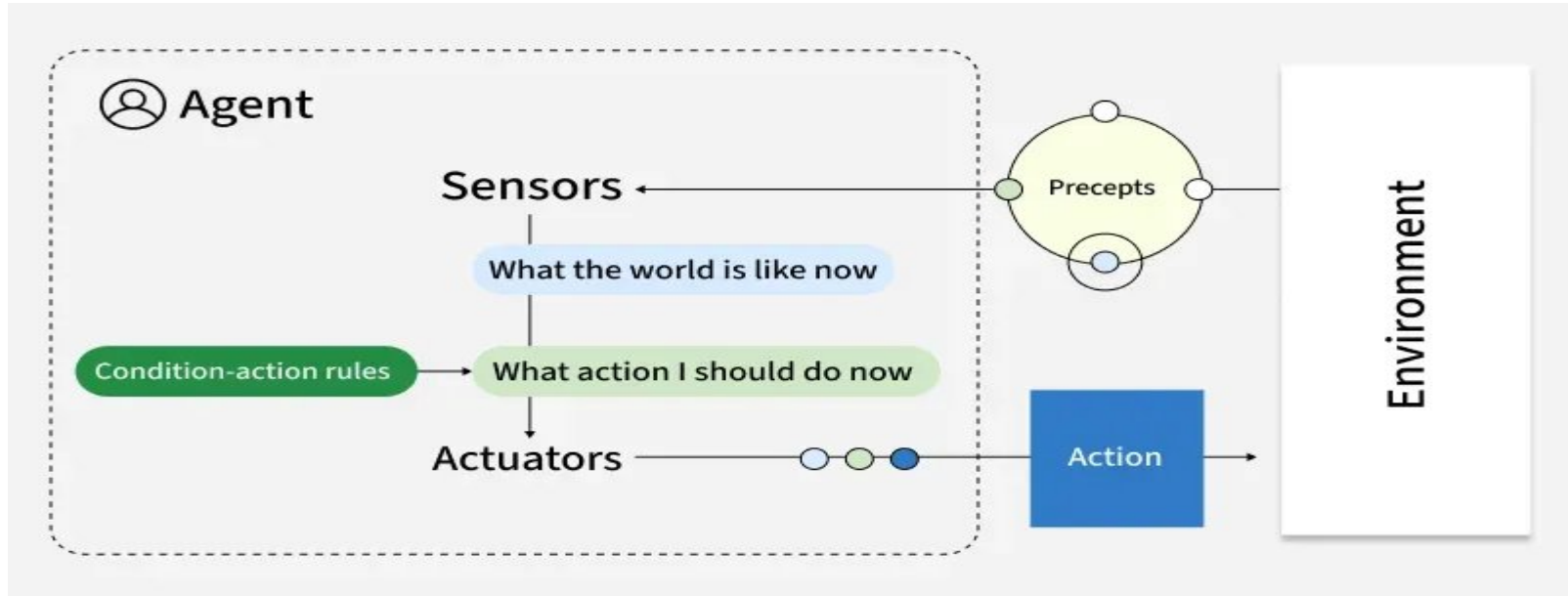
- **Profiling Module:** This module helps the agent understand its role and purpose. It gathers information from the environment to form perceptions. For example: A self-driving car uses sensors and cameras to detect obstacles.
- **Memory Module:** The memory module enables the agent to store and retrieve past experiences. This helps the agent learn from prior actions and improve over time. For example: A chatbot remembers past conversations to give better responses.
- **Planning Module:** This module is responsible for decision-making. It evaluates situations, weighs alternatives and selects the most effective course of action. For example: A chess-playing AI plans its moves based on future possibilities.

Architecture of AI Agents

- **Action Module:** The action module executes the decisions made by the planning module in the real world. It translates decisions into real-world actions. For example: A robot vacuum moves to clean a designated area after detecting dirt.

Types of Agents

1. Simple Reflex Agents



➤ Simple Reflex Agents

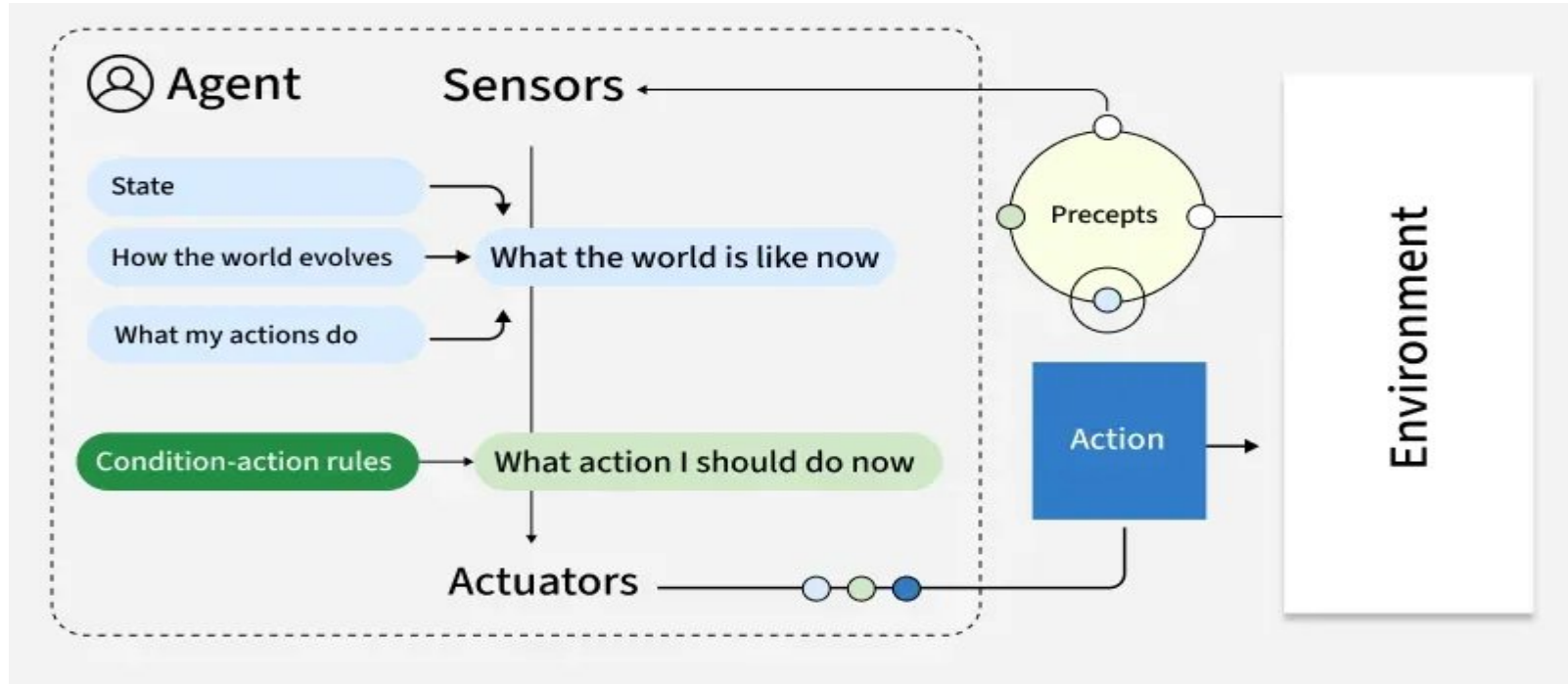
Simple reflex agents act based solely on current perceptions using condition-action rules. These agents respond directly to stimuli without considering past experiences or potential future states. They operate on basic "if-then" logic: if a specific condition is detected, execute a corresponding action.

Key Features:

- No memory of past states
- No model of how the world works
- Purely reactive behavior
- Function best in fully observable environments

For Example, Traffic light control systems that change signals based on fixed timing.

2. Model-Based Reflex Agents



Model-based reflex agents

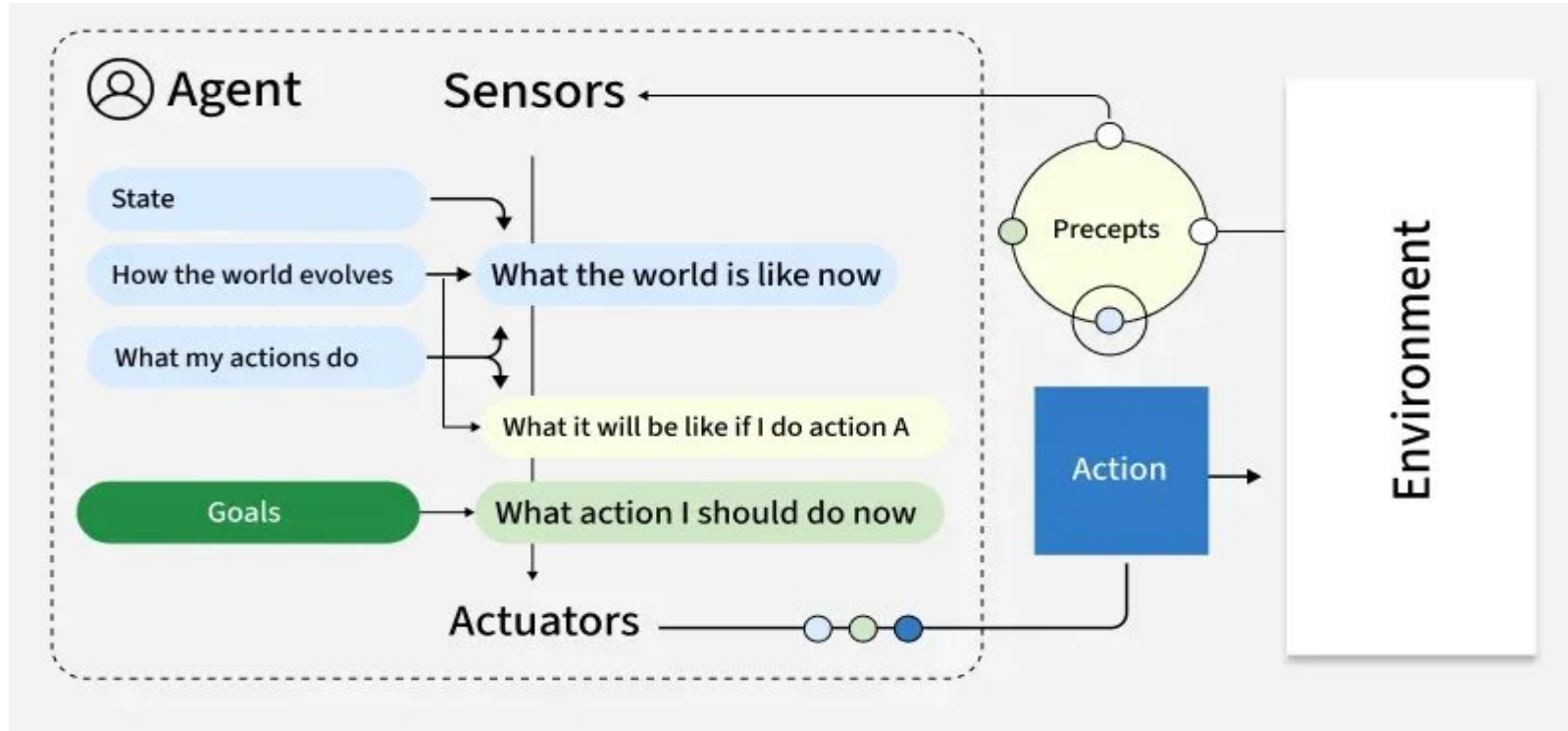
Maintain an internal representation of the world, allowing them to track aspects of the environment they cannot directly observe. This internal model helps them make more informed decisions by considering how the world evolves and how their actions affect it.

Key Features:

- ✓ Track the world's state over time
- ✓ Infer unobserved aspects of current states
- ✓ Function effectively in partially observable environments
- ✓ Still primarily reactive, but with contextual awareness

For example, Robot vacuum cleaners that map rooms and tracks cleaned areas.

3. Goal-Based Agents



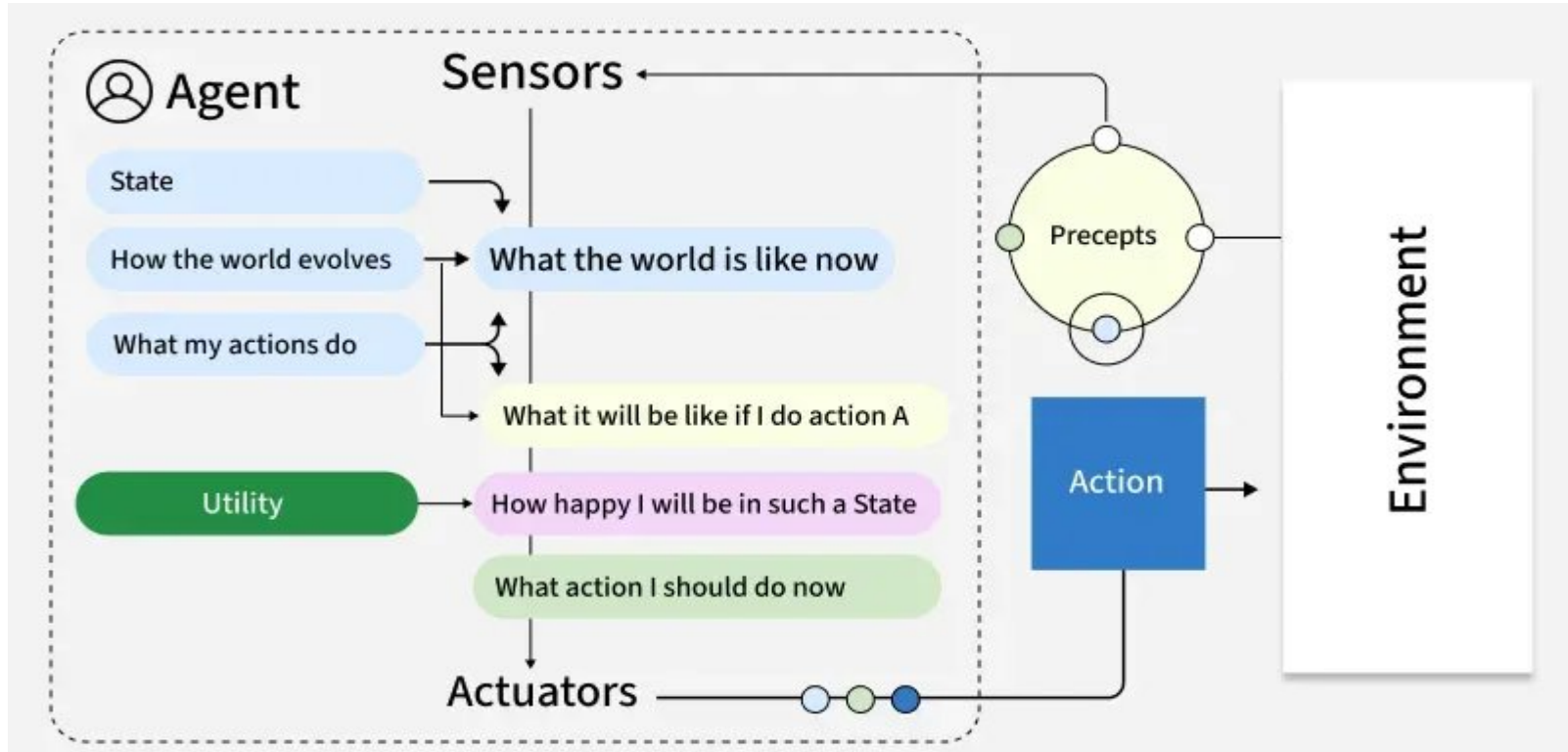
Goal-based agents plan their actions with a specific objective in mind. Unlike reflex agents that respond to immediate stimuli, goal-based agents evaluate how different action sequences might lead toward their defined goal, selecting the path that appears most promising.

Key Features:

- ✓ Employ search and planning mechanisms
- ✓ Evaluate actions based on their contribution toward goal achievement
- ✓ Consider future states and outcomes
- ✓ May explore multiple possible routes to a goal

For example, Logistics routing agents that find optimal delivery routes based on factors like distance and time. They continually adjust to reach the most efficient route.

4. Utility-Based Agents



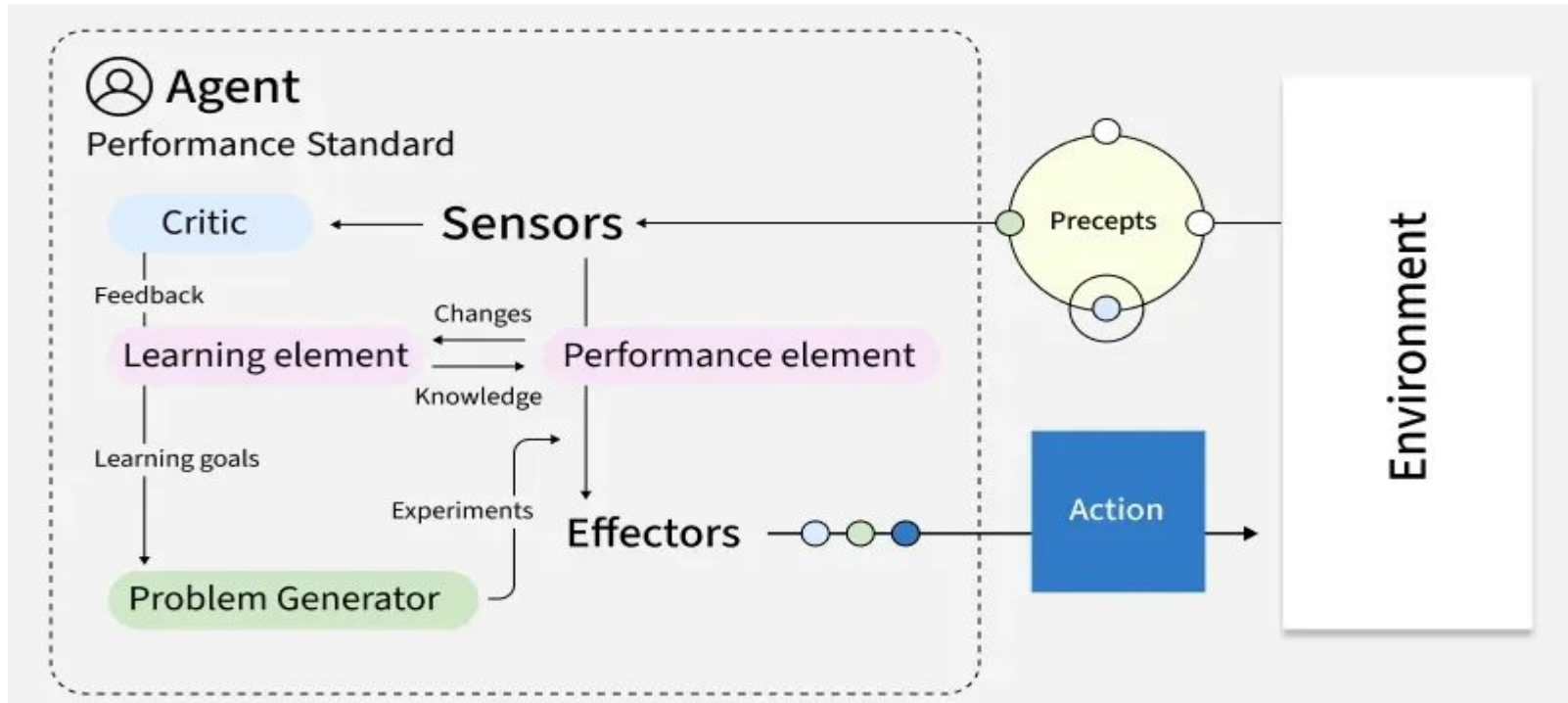
Utility-based agents extend goal-based thinking by evaluating actions based on how well they maximize a utility function—essentially a measure of "happiness" or "satisfaction." This approach allows them to make nuanced trade-offs between competing goals or uncertain outcomes.

Key Features:

- Balance multiple, sometimes conflicting objectives
- Handle probabilistic and uncertain environments
- Evaluate actions based on expected utility
- Make rational decisions under constraints

For example, Financial portfolio management agents that evaluate investments based on factors like risk, return and diversification operate by choosing options that provide the most value.

5. Learning Agents



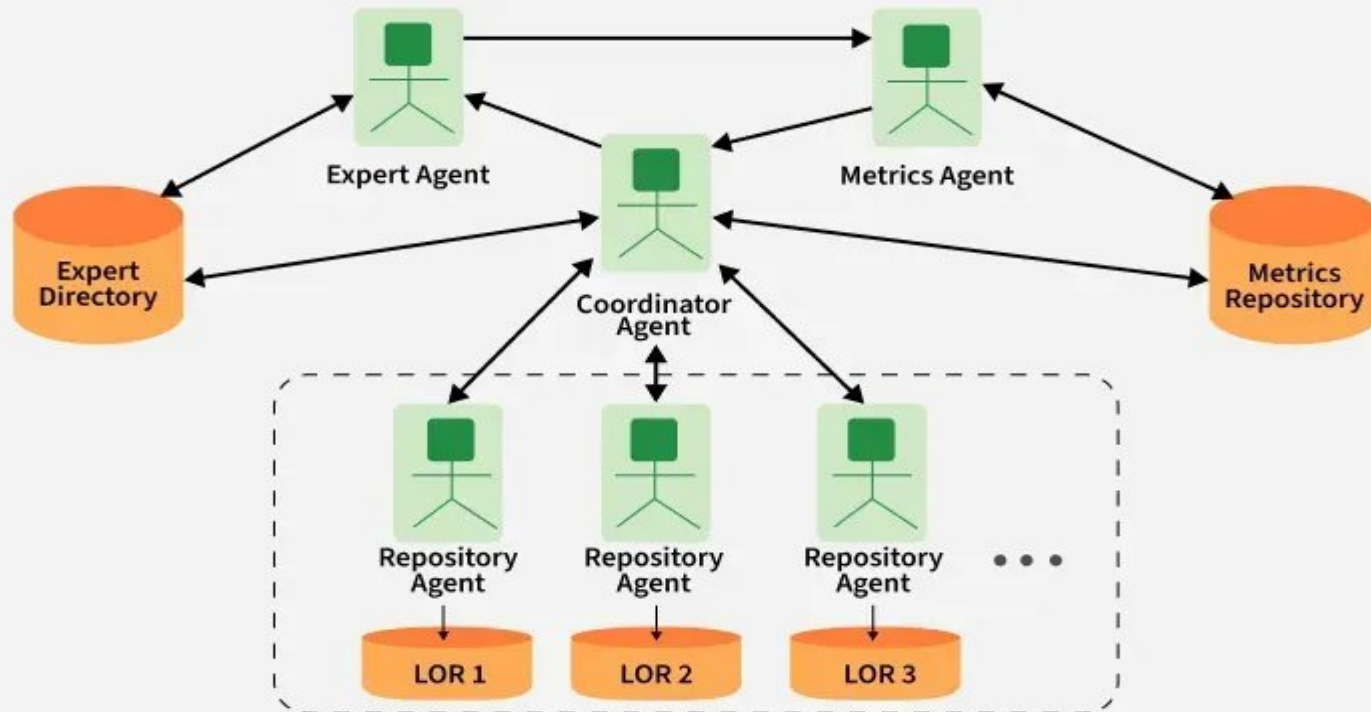
Learning agents improve their performance over time based on experience. They modify their behavior by observing the consequences of their actions, adjusting their internal models and decision-making approaches to achieve better outcomes in future interactions.

Key Features:

- Adapt to changing environments
- Improve performance with experience
- Contain both a performance element and a learning element
- Generate new knowledge rather than simply applying existing rules

For example, Customer service chatbots can improve response accuracy over time by learning from previous interactions and adapting to user needs.

6. Multi-Agent Systems (MAS)



Multi-agent systems consist of multiple autonomous agents that interact with each other within an environment. These agents may cooperate toward common goals, compete for resources or exhibit a mix of cooperative and competitive behaviors.

Types of multi-agent systems:

- Cooperative MAS: Agents work together toward shared objectives.
- Competitive MAS: Agents pursue individual goals that may conflict.
- Mixed MAS: Agents cooperate in some scenarios and compete in others.

Key Features:

- Agents act independently and control their own state.
- Agents align, collaborate or compete to reach goals.
- The system remains resilient if individual agents fail.
- Decisions are distributed; there's no single controller.



For example, a warehouse robot might use:

- Model-based reflexes for navigation
- Goal-based planning for task sequencing
- Utility-based decision-making for prioritizing tasks
- Learning capabilities for route optimization

Simulator

- A *simulator* or *solver* is an algorithm that computes the evolution of the state and outputs of a dynamical system .
- The set of trajectories followed by the state and outputs is known as the behaviour trace of the system.
- System-level simulation can be addressed following several approaches; most of them can be categorized into two major groups: monolithic simulation and co-simulation.
- The difference between these lies in the way in which dynamical systems are formulated, solved, and integrated.

Monolithic simulation

- ✓ Monolithic simulation consists in describing the response of all the components in the system to be studied with a single, all-encompassing set of equations.
- ✓ When this approach is followed, a single solver takes care of the integration of the dynamics of every component in the assembly.
- ✓ This solver typically has access to the details that describe the dynamic model of each component, as these are often required to build and solve their dynamics equations.

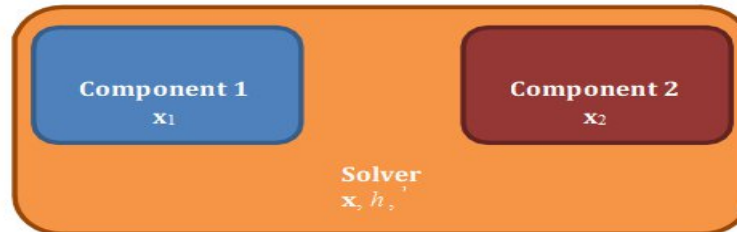


Figure : Scheme of a monolithic simulation.

- Figure shows the generic scheme of a monolithic simulation environment with two subsystems.
- Each component can be understood as a dynamical system with its own state, x_1 for component 1 and X_2 for component 2.
- A single solver, with a given integration method, is responsible for the assembly of the equations of motion and their numerical integration, often using a common integrator for all the components.
- The overall assembly is regarded as a single dynamical system; its state x includes the states X_1 and X_2 of the individual components, and a common evolution rule governs the change of all of them as time progresses.
- monolithic integration methods tend to be robust and efficient. They have been successfully used in a wide range of applications, including mechatronics and hydraulically actuated mechanical systems.

- Among their **drawbacks** is the need to develop a dynamic formulation that is able to deal with the behaviour of every subsystem, which can become a complicated task when these are governed by very dissimilar physical equations.
- Besides, in many cases the solver needs to have full access to the implementation details of each component to assemble the dynamics equations of the whole system.
- Because of confidentiality reasons, this poses a problem in industrial applications that combine components from different vendors.
- An additional difficulty in industry stems from the fact that, sometimes, companies use for their simulation tasks software models that have been developed over the years with particular modelling tools. Repeating the development of these models to integrate them in a new monolithic simulation software would be costly and time-consuming.
- Co-simulation is an alternative to monolithic implementations that offers a possible answer to the above mentioned problems.

Co-simulation

- Co-simulation is the joint simulation of loosely coupled stand-alone sub-simulators.
- A co-simulation algorithm takes care of time synchronization and interactions across the sub-simulators.
- The interactions between these sub-simulators are only synchronized at discrete communication points.
- Sub-simulators are assumed to be completely independent of each other between the communication points.
- The sub-simulators behave conceptually like black boxes.

Co-simulation

- They accept inputs (from other sub-simulators), advance in time with a **built-in solver** routine up to the next communication point, and finally output some results.
- The results may in turn be used again as inputs to other sub-simulators.

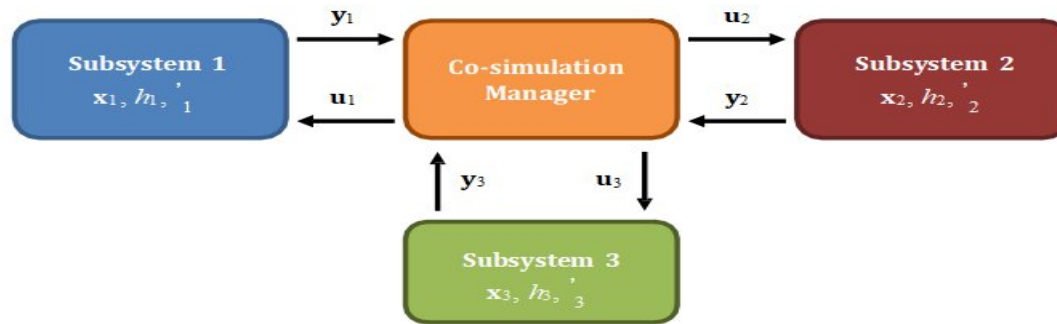


Figure : Scheme of a generic co-simulation environment.

A generic co-simulation scheme with three subsystems . Each subsystem is an independent solver, with its own state x_i ($i = 1, 2, 3$), integration method i , and step-size h_i .

The interaction of each solver with its environment, i.e., the rest of subsystems in the assembly, is represented by a set of coupling variables that includes its input u_i and output y_i .

The exchange of coupling variables between the subsystems is coordinated by a co-simulation manager or orchestrator, responsible for the synchronization of the integration processes of the solvers.

The exchange of coupling variables between the subsystems takes place at discrete-time communication points, at which they receive their input and send their output to the manager.

Between two consecutive communication points, each subsystem proceeds with the integration of its state without any interaction with its environment.

This means that information about the evolution of the rest of subsystems is generally not available until the next communication point is reached. The time interval between two communication points is known as a macro time-step.

Co-simulation processes usually consist in two stages. During the initialization phase, a consistent initialization of the simulators must be ensured.

The initial states of the different subsystems may have to satisfy one or more algebraic constraints, which generally involve the inputs and outputs of the system and can be expressed as

$$\Phi \mathbf{x}_{1,0}, \dots, \mathbf{x}_{n,0}, \mathbf{y}_{1,0}, \dots, \mathbf{y}_{n,0}, \mathbf{u}_{1,0}, \dots, \mathbf{u}_{n,0} = \mathbf{0}$$

where n is the total number of subsystems in the setup. Solving Eq. may require an iterative procedure to ensure the consistency of the initial states of all the subsystems. Once the initialization is complete, the execution phase can begin.

Co-simulation schemes -

2.1 Continuous and event-driven co-simulation -

When a subsystem in a co-simulation environment has its evolution rule described by differential equations it is called a continuous subsystem. Such a system can be described with the following initial value problem.

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$$

$$\mathbf{y} = \mathbf{g}(\mathbf{x}, \mathbf{u})$$

$$\mathbf{x}(t = 0) = \mathbf{x}_0$$

where $\mathbf{x}$ is the system state, $\mathbf{u}$ and $\mathbf{y}$ stands for the inputs and the outputs, respectively, and $\mathbf{f}$ and $\mathbf{g}$ are, in general, nonlinear functions. Simulation tools often integrate the differential equation in (15) by means of a time discretization. For instance, with a Taylor series expansion

$$\mathbf{x}(t + h) = \mathbf{x}t + h \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) + \boldsymbol{\eta}$$

where h is the integration step-size and $\boldsymbol{\eta}$ is the approximation error incurred by the discretization with respect to time. The behaviour trace obtained this way is an approximation of the one that the system would theoretically have. From this point of view, most time-driven subsystems can be considered continuous subsystems.

A *discrete-event subsystem*, on the other hand, has its change of state triggered by an external event. These subsystems feature a limited set of possible states, input events, and output events.

While continuous subsystems have states that evolve continuously over time and outputs that have to obey the physical laws of continuity, discrete-event subsystems can have states that assume multiple values at the same time (*transiency*) and discontinuous outputs.

The term **hybrid co-simulation** refers to co-simulation environments in which some subsystems can be considered continuous and others are discrete-event ones. There are two general approaches to tackling these problems. A possibility is formulating the continuous subsystems as discrete-event ones, and using a discrete-event manager to orchestrate the simulation. Conversely, the discrete-event subsystems can be converted into continuous subsystems; in this case a continuous manager will be used to coordinate the numerical integration.

Implementation

Implementation is a crucial step in co-simulation applications.

In general, a certain co-simulation scheme can be implemented using a wide range of implementation techniques, which will have an impact on the overall performance and stability of the numerical integration.

The different solvers in a co-simulation environment may be executed in the same computer or over a network, employing homogeneous or heterogeneous computer architectures.

In certain co-simulation applications, some components are physical, non-simulated elements, that interact with the virtual ones represented by the simulators.

The implementation techniques to be used will need to address specific requirements depending on the configuration of the simulation environment.

In all cases, a necessary requirement to couple different solvers in a coordinated integration process is that the exchange of information between them takes place through a protocol that all of them can understand and use to communicate with their environment.

The FMI standard

When coupling subsystems in a co-simulation environment, it is desirable that the communication between solvers takes place via an unambiguous interface with minimal specifications and impact on the internals of the subsystems. This interface must be general enough to be used with subsystems of all natures and physical behaviours, so that it can be widely accepted both in academic and industrial environments .

The Functional Mock-up Interface (FMI) was conceived to address these needs: it is a tool independent standard to support both model exchange and co-simulation of dynamic models using a combination of XML files and compiled C code.



Thank You